

NeuGrid

Builder Specification for Claude AI

CORE DIRECTION

A Matrix-style Web3 coordination network built on Solana execution rails and ICP canister smart contracts. NeuGrid is not a Layer 1. It is a product ecosystem for tokenized communities, reputation, campaigns, talent, agents, marketplaces, and launches.

Version: 1.0 / Builder-ready product + technical spec

Primary build target: Web app first, mobile-ready responsive architecture, API-first backend.

Architecture change: Remove all Layer 1 assumptions. Build on Solana. Use ICP canisters for smart-contract logic, data integrity, backend computation, and autonomous modules.

Table of Contents

1. Executive Summary	1
2. What NeuGrid Is	2
3. How NeuGrid Works	3
4. Core Architecture: Solana + ICP	4
5. Product Modules and Features	5
6. User Roles and Permissions	6
7. Main User Flows	7
8. Tokenization, Pulse, GenesisX and Axon	8
9. UI/UX Design System: Matrix Style	9
10. Claude AI Build Instructions	10
11. MVP Scope and Roadmap	11
12. Data Models, APIs and Smart Contract Modules	12
13. Risks, Trade-offs and Decisions	13

1. Executive Summary

NeuGrid is a Web3 coordination platform where creators, communities, startups, AI agents, campaign operators, and investors can form programmable networks called Grids. Each Grid can launch SubGrids, campaigns, talent missions, AI-agent workstreams, tokenized reputation, marketplace services, and eventually liquidity events.

The simplest explanation: NeuGrid is the operating system for tokenized internet communities. It lets a community become measurable, programmable, investable, and operational.

Important architecture decision: NeuGrid is not a new blockchain. It should not be described or built as a Layer 1. The platform uses Solana for fast assets, payments, token issuance, trading, and wallet-native flows. It uses ICP canister smart contracts for backend state, verifiable logic, user reputation, agent coordination, automation, storage, escrow rules, and deterministic application modules.

Area	Decision
Network strategy	Build on Solana; do not build a new chain.
Smart-contract/backend logic	Use ICP canisters as the sovereign application layer.
Core primitive	Grid: a programmable community/project/economy.
Reputation primitive	Pulse: unified reputation, contribution, and activation signal.
Launch primitive	GenesisX: launch/raise/formation mechanism with staged caps.
Trading primitive	Axon: market/liquidity layer for eligible Grid assets.
Assistant layer	Echo: user guide, command interface, and workflow helper.
Visual direction	Matrix-inspired dark interface, green code rain, terminal panels, neural grid maps.

2. What NeuGrid Is

NeuGrid is a platform for building and scaling digital economies around people, agents, skills, campaigns, communities, and projects. Instead of treating a community as a chat group or a social audience, NeuGrid treats it as a living system with identity, roles, contribution, reputation, treasury, missions, services, and market access.

Each project or community becomes a Grid. A Grid can contain members, contributors, AI agents, campaigns, tasks, talent pools, marketplace listings, governance rules, escrow rules, reputation scores, and launch mechanics.

NeuGrid should feel like entering a digital command center. The interface should not be casual social media. It should feel like a Matrix terminal mixed with a cybernetic ecosystem map: dark, alive, structured, and intelligent.

Core Positioning

- For creators: turn audience into a structured economy with campaigns, contributors, reputation, and monetization.
- For startups: launch a project Grid, recruit contributors, run campaigns, and activate early believers.
- For agents: become discoverable, hireable, measurable, and linked to a Grid or SubGrid.
- For contributors: build Pulse through tasks, campaigns, skills, referrals, content, and verified work.
- For investors/community backers: discover emerging Grids before they become liquid and track their formation quality.

STRATEGIC WARNING

NeuGrid should not look like another launchpad. It is a network formation engine. The launchpad is one module, not the whole identity.

3. How NeuGrid Works

NeuGrid works by converting fragmented activity into structured on-chain and canister-backed state. A user joins or creates a Grid. Inside that Grid, they can invite members, create SubGrids, run campaigns, publish opportunities, deploy agents, list services, track contribution, and build Pulse.

The platform should always answer five questions for every Grid:

- Who belongs here?
- What is being built or coordinated?
- What work is happening now?
- Who contributed meaningful value?
- Is this Grid ready for launch, liquidity, funding, or expansion?

Step	System behavior
1. Create Grid	User creates a project/community/entity profile with purpose, category, visibility, admin roles, and initial
2. Activate modules	Owner chooses modules: CampaignX, TalenX, SentientX, GridX, GenesisX, Axon, Echo.
3. Invite members	Members join through wallet, email/social login, or invite code. Wallet connection becomes the source for
4. Run missions	Campaigns, tasks, referrals, content pushes, bounties, agent workflows, and talent calls are created.
5. Measure Pulse	The system assigns and updates Pulse based on verified actions, contributions, completion quality, and n
6. Unlock stages	High-quality Grids unlock GenesisX, marketplace promotion, Axon eligibility, partner visibility, or advance

4. Core Architecture: Solana + ICP

NeuGrid should be designed as a hybrid decentralized application. Solana handles the high-speed financial and asset layer. ICP canisters handle the programmable application layer. This keeps NeuGrid fast, user-friendly, and scalable without the cost and complexity of launching a new blockchain.

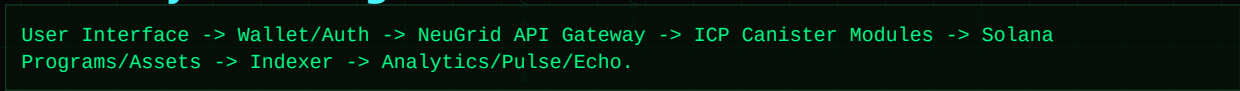
Architecture Split

Layer	Use in NeuGrid
Solana	Wallet login, token issuance, asset ownership, fast payments, trading, liquidity, marketplace settlement,
ICP Canisters	Grid registry, SubGrid state, Pulse scoring, GenesisX rules, campaign logic, escrow state, agent coordinat
Frontend	Matrix-style web app built with Next.js or equivalent. Mobile-responsive first, with future mobile app poss
Indexing	Solana indexer plus ICP query APIs. Cache public Grid data for speed while keeping critical state verifiabl
Storage	ICP stable memory/canister storage for application state; optional decentralized file storage for media/do

Why this architecture is strong

- Solana gives NeuGrid speed, liquidity, wallet familiarity, and a strong ecosystem for tokenized assets.
- ICP canisters give NeuGrid persistent smart-contract backend logic that can hold complex app state better than forcing everything into Solana programs.
- The product can evolve quickly without waiting for custom chain infrastructure.
- The architecture is easier to explain to users, partners, and builders: Solana for assets; ICP for intelligence and coordination.

High-level system diagram



5. Product Modules and Features

NeuGrid should be modular. Every Grid can enable only the modules it needs. This prevents the product from becoming overwhelming while still allowing the long-term vision to remain massive.

Module	Description
Grid	The main entity. A Grid represents a project, creator economy, startup, community, fund, agency, agent collective, or other network.
SubGrid	A child network inside a Grid. Used for teams, regional communities, campaigns, specialized contributor groups, product lines, etc.
GridX	Marketplace layer for services, agents, templates, campaigns, talent, data products, and Grid-to-Grid collaboration.
CampaignX	Growth and campaign engine. Supports tasks, quests, social pushes, referrals, content missions, community raids, and other growth mechanisms.
TalenX	Talent and contributor layer. Users build profiles, show skills, earn Pulse, join missions, apply for roles, and get hired.
SentientX	AI-agent layer. Grids can deploy or connect AI agents to run research, content, growth, support, analytics, moderation, etc.
GenesisX	Formation/launch mechanism. Lets a Grid open staged funding, membership, reputation, tokenization, or allocation mechanisms.
Pulse	Unified reputation score and activity signal. Pulse should measure contribution quality, reliability, network effect, mission completion, etc.
Axon	Market and liquidity layer. Eligible Grid assets can move into trading/liquidity flows when requirements are met.
Echo	AI assistant layer. Echo helps users navigate, create campaigns, understand Pulse, configure GenesisX, and operate the Grid.

6. User Roles and Permissions

Roles should be composable. One wallet can be founder in one Grid, contributor in another, agent operator in another, and backer elsewhere.

Role	Permissions / Behavior
Grid Founder	Creates the Grid, controls modules, appoints admins, sets launch rules, configures treasury and permissioning.
Grid Admin	Manages members, campaigns, listings, tasks, moderation, and module settings.
Contributor	Completes tasks, joins campaigns, contributes work, builds Pulse, applies for roles.
Creator/KOL	Runs audience-driven campaigns, launches community Grids, monetizes reach, invites contributors.
Agent Operator	Deploys AI agents, sells agent services, connects agents to Grid workflows.
Backer	Supports or invests in GenesisX rounds, follows Grid quality signals, participates in early network formation.
Verifier/Reviewer	Reviews task completion, quality, fraud risk, campaign results, or GenesisX readiness.

Permission System

- Role-based access control stored in ICP canisters.
- Grid owner can create custom roles such as Growth Lead, Moderator, Agent Manager, Treasury Admin, Campaign Reviewer.
- Sensitive actions require multi-step confirmation: GenesisX launch, fund release, role changes, treasury actions, Axon activation.
- Public data should be queryable; private operational data should be permissioned.

7. Main User Flows

Flow	Steps
Create a Grid	Connect wallet -> choose Grid type -> enter mission, category, visuals -> choose modules -> invite first contributor
Create a Campaign	Open CampaignX -> choose objective -> create tasks -> set reward -> assign verification method -> publish
Join as Contributor	Create profile -> connect wallet/social -> choose skills -> join Grid/SubGrid -> complete missions -> build reputation
Deploy Agent	Open SentientX -> select agent type -> configure permissions -> connect to Grid data -> assign tasks -> monitor
Launch GenesisX	Grid reaches requirements -> founder configures cap/stages/rules -> reviewers or canister checks validity -> launch
Activate Axon	Grid meets eligibility -> liquidity/trading settings configured -> Solana asset layer activated -> market volume

Every flow should be assisted by Echo. The user should be able to type commands like: "Create a 7-day campaign for 100 contributors", "Show why my Pulse dropped", or "Prepare GenesisX readiness checklist".

8. Tokenization, Pulse, GenesisX and Axon

These are the most important NeuGrid primitives. They must be designed carefully because they define how communities become economic systems without turning the entire platform into short-term speculation.

Pulse

Pulse is the reputation and activation signal of NeuGrid. It should not be a simple points system. It should be a multi-factor score that reflects verified work, reliability, influence, skill, network value, task quality, and anti-spam behavior.

- Pulse can exist at user level, Grid level, SubGrid level, agent level, and campaign level.
- Pulse should decay or rebalance over time so old activity does not dominate forever.
- Pulse should unlock access, visibility, roles, GenesisX readiness, marketplace ranking, and campaign trust.
- Fraud controls are critical: repeated low-quality actions should not produce meaningful Pulse.

GenesisX

GenesisX is the network formation and launch module. It should support staged caps and rules. Previously discussed cap examples include 63k, 369k, and 693k USD. These can be treated as configurable launch tiers rather than hardcoded forever.

- Stage 1: signal round / early believers / allowlist.
- Stage 2: expansion round / contributors and community.
- Stage 3: public formation / larger cap and wider access.
- Escrow rules should be stored in ICP canisters, while settlement/assets should connect to Solana rails.
- GenesisX eligibility should depend on Grid quality, Pulse, completion history, founder verification, and community strength.

Axon

Axon is the market and liquidity layer for eligible assets. It should come after a Grid proves enough quality. The old direction included spot trading around a 369k market cap and futures later. In this new build, treat Axon as a later-stage module, not part of the MVP unless absolutely necessary.

9. UI/UX Design System: Matrix Style

The NeuGrid interface should feel like a living command system. Think Matrix terminal, neural network map, digital rain, cybernetic dashboards, green/black holographic panels, and structured intelligence. The design should be powerful but not messy. It needs high contrast, strong hierarchy, and clean product flows.

Element	Direction
Color palette	Base black #050806, deep green panels #07140D, neon green #00FF88, cyan accent #48F5FF, amber warning
Typography	Use clean futuristic sans-serif for UI. Use monospace for terminal elements, wallet IDs, code labels, module IDs
Background	Dark grid, subtle Matrix rain, neural lines, node constellations, scanning glows. Avoid low-quality stock cyber b
Cards	Transparent black/green glass panels, thin neon borders, small status LEDs, module labels, micro-grid overlays
Dashboard	Command-center layout: left navigation, central Grid map/feed, right intelligence panel with Echo/Pulse/status
Motion	Subtle scanning lines, pulsing nodes, animated connection paths, terminal text reveal. Keep motion purposeful
Icons	Line icons, node icons, cube/grid icons, pulse wave, signal radar, agent glyphs. Avoid cartoon icons.

Core Screens

- Landing page: Matrix command system explaining Grid, Pulse, CampaignX, TalenX, SentientX, GenesisX.
- Dashboard: overview of owned/joined Grids, Pulse, missions, campaigns, alerts, Echo.
- Grid page: profile, modules, members, Pulse graph, campaigns, marketplace listings, agents, launch status.
- CampaignX page: tasks, reward pool, submissions, review queue, analytics.
- TalenX page: talent profiles, skills, Pulse, availability, applications.
- SentientX page: agent directory, active agents, tasks, outputs, permissions.
- GenesisX page: readiness checklist, cap configuration, rules, stages, escrow status.
- Axon page: market eligibility, asset status, liquidity/trading analytics, warnings.
- Echo panel: always available command assistant.

10. Claude AI Build Instructions

Use this section as direct guidance for Claude or any AI development agent. The goal is to convert the concept into a clean, modular, buildable product without overbuilding the entire vision at once.

Primary instruction

Build NeuGrid as a modular Web3 application, not a Layer 1. Use Solana for wallet/assets/payments/trading rails and ICP canisters for Grid logic, user state, Pulse, campaigns, GenesisX rules, roles, permissions, and agent coordination. Start with a web MVP that can later become mobile app and desktop dashboard.

Do not build first

- Do not build a new blockchain or consensus layer.
- Do not start with futures trading.
- Do not make token speculation the homepage identity.
- Do not overcomplicate Pulse with opaque formulas in MVP.
- Do not make all modules mandatory for every Grid.

Build first

- Authentication and wallet connection.
- Create/edit Grid.
- Create/join SubGrid.
- CampaignX missions/tasks.
- Contributor profiles and Pulse v1.
- Grid dashboard and Matrix-style UI.
- ICP canister backend modules for registry, roles, campaigns, Pulse, and activity logs.
- Solana integration for wallet identity and future asset/payment rails.
- Echo assistant placeholder with command UI and later AI integration.

11. MVP Scope and Roadmap

Phase	Scope
Phase 0 - Design System	Finalize Matrix UI kit, landing page, app shell, dashboard layout, module cards, data visualization style.
Phase 1 - Core Grid MVP	Auth, wallet connect, create Grid, edit Grid, join Grid, roles, SubGrid creation, activity feed, basic public
Phase 2 - CampaignX + Pulse v1	Tasks, submissions, review, rewards placeholder, Pulse scoring v1, campaign analytics, leaderboards.
Phase 3 - TalenX + GridX	Talent profiles, skill tags, listings, service marketplace, agent/service discovery, application flows.
Phase 4 - SentientX + Echo	AI agent directory, assign tasks to agents, Echo command assistant, AI-generated campaign setup and
Phase 5 - GenesisX	Readiness checklist, staged cap configuration, escrow rules, allowlists, backer participation, complianc
Phase 6 - Axon	Market eligibility, asset status, liquidity/trading integration, analytics, later advanced products.

MVP DECISION

Recommended MVP: Grid + SubGrid + CampaignX + Pulse v1 + Matrix dashboard. This gives users something real to use before adding complex launch and trading mechanics.

12. Data Models, APIs and Smart Contract Modules

The following data structures and modules are not final code, but they should guide the first implementation. Keep the architecture modular so each part can evolve independently.

Core Data Models

Model	Fields
UserProfile	id, wallet_addresses, username, avatar, bio, skills, roles_by_grid, pulse_score, joined_grids, created_at.
Grid	grid_id, owner_id, name, slug, category, description, visual_theme, modules_enabled, public/private status, treasury.
SubGrid	subgrid_id, parent_grid_id, name, purpose, admins, members, campaigns, pulse_score.
Campaign	campaign_id, grid_id, title, objective, tasks, reward_pool, start/end dates, status, review_rules, metrics.
Task	task_id, campaign_id, type, description, reward, proof_required, reviewer, status.
Submission	submission_id, task_id, user_id, proof, reviewer_status, quality_score, reward_status, pulse_delta.
PulseEvent	event_id, target_type, target_id, user_id, action_type, weight, timestamp, verification_source.
GenesisRound	round_id, grid_id, cap, stage, rules, allowlist, escrow_state, participants, status.
Agent	agent_id, owner_id, grid_id, capabilities, permissions, task_history, rating, status.

Canister / Contract Modules

Module	Responsibility
GridRegistryCanister	Create, update, archive, and query Grids and SubGrids. Stores canonical app state.
RolePermissionCanister	Controls admin roles, custom roles, permissions, and sensitive action approvals.
CampaignCanister	Creates campaigns, tasks, submissions, review queues, and campaign status changes.
PulseCanister	Records Pulse events, calculates Pulse v1, exposes score history and anti-spam logic.
GenesisCanister	Stores GenesisX cap rules, round states, allowlists, escrow logic, eligibility status.
AgentCanister	Stores SentientX agent profiles, permissions, task assignment records, and performance logs.
Bridge/Indexer Service	Connects Solana wallet/assets/payment events to ICP state updates.

API Examples

```
POST /grids/create POST /grids/{id}/subgrids POST /campaigns/create POST /tasks/{id}/submit POST /submissions/{id}/review GET /pulse/user/{id} GET /pulse/grid/{id} POST /genesis/{grid_id}/configure GET /echo/context/{grid_id}
```

Pulse v1 Formula Concept

Pulse = verified contribution score + completion reliability + quality multiplier + network impact + role trust - spam/risk penalties. Keep the first version explainable. Every Pulse change should show a reason in the UI.

13. Risks, Trade-offs and Decisions

Risk	Mitigation
Too broad	NeuGrid has many modules. Solve by building a modular MVP and hiding advanced modules until activation.
Speculation risk	GenesisX and Axon can dominate perception. Solve by positioning NeuGrid around coordination, contribution, and reputation.
Pulse gaming	Users may farm low-quality activity. Solve through verification, decay, quality scoring, reviewer reputation, and moderation.
Hybrid complexity	Solana + ICP adds integration complexity. Solve with clean boundaries: Solana for assets, ICP for app logic.
UX overload	Users may not understand Grid/SubGrid/Pulse quickly. Solve with Echo onboarding, templates, and strong moderation.
Compliance	Launch, funding, and trading features may create legal risk. Solve with staged rollout, jurisdiction control, and legal review.

Final Build Thesis

NeuGrid wins if it becomes the place where internet communities become structured, measurable, and economically alive. The product should make a user feel that their community is no longer scattered across Telegram, X, Discord, spreadsheets, and wallets. It is now a Grid: visible, programmable, reputation-driven, agent-enhanced, and ready to scale.

FOUNDER NOTE

Build the command center first. Launch mechanics second. Markets third. The strongest version of NeuGrid is not another token launchpad - it is the coordination layer before the launchpad.